

dzo

Install, update and restart DayZ servers on one host, with rootless podman and btrfs snapshots.

■ reads ■ changes something ■ destroys data

1 Install the package

Creates the dayz user and /var/lib/dzo. Put that directory on btrfs if you want snapshots. Everything else runs as dayz.

<code>sudo apt install ./dzo_<version>_amd64.deb</code>	install, or upgrade
<code>sudo -iu dayz</code>	work as the service user

2 Point dzo at your site repository

The site repository holds every instance's settings. Create an empty git repository, then set it in /etc/dzo/config.yaml:

```
site:
  remote: git@host:you/dayz-site.git
  branch: main
```

<code>dzo setup</code>	directories, container images, site checkout
<code>dzo site validate</code>	load the site, resolve every instance

3 Log in to Steam, download the server

<code>dzo steam login --user <account></code>	password and Steam Guard, asked once
<code>dzo product install dayz-stable</code>	about 3 GB (dayz-experimental works the same)

4 Describe the instance, then push it

Add instances/chernarus/instance.yaml and a serverDZ.cfg next to it. Set a join password in the config.

```
name: chernarus
product: dayz-stable
map: dayzOffline.chernarusplus # Bohemia's mission
ports: {game: 2302, rcon: 2303, query: 27016}
network: host
mods: []
restarts: {schedule: ["*-*-* 00/4:00"]} # every 4 h
```

<code>dzo site pull</code>	fetch what you pushed
----------------------------	-----------------------

5 Create it and start it

<code>dzo instance render chernarus --dry-run</code>	preview what will be written
<code>dzo instance create chernarus</code>	data directory, mission, first render
<code>dzo instance apply chernarus</code>	write the units and timers
<code>dzo start chernarus</code>	returns when the server answers Steam queries
<code>dzo status</code>	state, players, health, restarts

6 Add mods

<code>dzo mod add 1559212036 --instance chernarus</code>	install a workshop mod, add it to the list
<code>dzo mod add <id> --instance chernarus --server</code>	loaded with -servermod (clients don't get it)
<code>dzo site commit -m "add mods" --push</code>	mod add edits your checkout, keep it in git
<code>dzo restart chernarus --minutes 5</code>	announce, lock, kick, restart with the mods

Where things live

/etc/dzo/config.yaml	host settings
/var/lib/dzo/site	your site checkout
.../instances/<n>	one server: mpmissions, profiles, storage
.../cache	server builds and mods, kept as generations
.../snapshots	btrfs backups
.../logs	archived profile logs
.../secrets	Steam session, RCon passwords

Units and logs

Each server is the user unit dzo-<n>.service. Use systemctl --user and journalctl --user on it as on any unit.

Ports

Give every instance its own **game**, **rcon** and **query** port. RCon listens on localhost only. Open the game and query ports in the firewall.

A new mod or build never reaches a running server on its own. It switches at the next restart.

Start, stop, look

dzo status <n>
state, players, health, pending updates

dzo start <n>
waits until the server answers

dzo stop <n>
asks over RCon, then stays stopped

dzo logs <n> -f
follow the server console

dzo instance ack-failure <n>
allow starts again after a failed render or a crash loop

Restart without annoying players

dzo restart <n> --minutes 30 --lock 3
announce every minute, lock 3 minutes before, kick, restart

dzo restart <n> --now
no countdown

dzo restart <n> --cancel
stop the countdown, unlock

dzo instance apply <n>
after changing restarts.schedule: rewrite the timers

Mods and updates

dzo instance mods list <n>
the list; its order decides which mod wins a merge

dzo instance mods move <id> --before <other>
change that order

dzo mod remove <id> --instance <n>
drop a mod from the list

dzo update check [--apply]
hourly by timer; policy: auto restarts for you

dzo mod refresh <id> --force
download again after a broken download

New server build

A new build is only reported. You switch each instance yourself.

dzo product update dayz-stable
download the new build

dzo instance upgrade <n> --build <id> --dry-run
plan and mod dependency check for that build

dzo instance upgrade <n> --build <id>
announce, stop, snapshot, switch, start. A failed switch reverts

Mission and config

dzo mission status <n>
source, age, what the next render changes

dzo mission update <n>
fetch a newer mission, show the plan

dzo instance render <n>
apply in place, only files dzo manages

dzo mission rollback <n> --list
managed files from file history

Backups

Snapshots are taken before upgrades, mod updates and wipes.

dzo backup create <n>
snapshot now

dzo backup list <n>
what exists, and why

dzo backup diff <n> <id>
files that changed since then

dzo backup pin <n> <id>
keep it, whatever the retention says

RCon and logs

dzo rcon console <n>
interactive, with connects and chat

dzo rcon exec --instance <n> players
one command

dzo rcon rotate <n> --restart
new RCon password, read at the next start

dzo logs rotate <n> --dry-run
which profile logs would be archived

dzo logs archive <n> --since 7d
list the archive

Look inside

dzo shell <n>
debug container with the instance's mounts, server not started

dzo exec <n> ls /profiles
a command in the running container

These delete data

They ask you to confirm (--yes skips it). wipe, reinit and restore take a snapshot first.

dzo wipe <n>
the world (persistence), not the mission

dzo mission reinit <n>
recreate the live mission from the original

dzo instance upgrade <n> --build <id> --wipe
switch builds and wipe the world

dzo restore <n> <id> [--path ...]
go back to a snapshot, all or one path

dzo instance remove <n>
data directory and, unless --keep-snapshots, its snapshots